国最大模型开源技术栈2万一M3单GPU跑满PagedAttention原理

小米哥

Figure 1. Time to first token on the single-GPU H100 with Llama-3-8B-Instruct.

注意：本文基于代码，如有错误请指正或留言。

目录： 国最大模型开源技术栈2万一M3单GPU跑满PagedAttention原理 Paged Attention 的核心思想 Paged Attention 的 Kernel 实现 Paged Attention 的 Kernel 调用 Paged Attention 的内存管理 Paged Attention 的调度 Paged Attention 的 Kernel 实现 Paged Attention 的 Kernel 调用 Paged Attention 的内存管理 Paged Attention 的调度 参考

1.Paged Attention 的核心思想

目录： 国最大模型开源技术栈2万一M3单GPU跑满PagedAttention原理 Paged Attention 的核心思想 Paged Attention 的 Kernel 实现 Paged Attention 的 Kernel 调用 Paged Attention 的内存管理 Paged Attention 的调度 Paged Attention 的 Kernel 实现 Paged Attention 的 Kernel 调用 Paged Attention 的内存管理 Paged Attention 的调度 参考

目录： 国最大模型开源技术栈2万一M3单GPU跑满PagedAttention原理 Paged Attention 的核心思想

- 修改成的token插入逻辑块：对于A1和A2来说，将其生成的token插入各自的逻辑块block1；
- 触发物理块copy-on-write机制：由于 fathers/mothers 是两个完全不同的token，因此对物理块block1触发复制机制，即在物理内存上新开辟一块空间，此时物理块block1只和A2的逻辑块block1映射，将其ref count减去1；物理块block3只和A1的逻辑块block1映射，将其ref count设为1。

总结起来，VLLM节省KV cache显存的核心思想是，对于相同数据对应的KV cache，能复用则尽量复用；无法复用时，再考虑开辟新的物理块。

4.2 Beam Search

我们从右往左来看这张图，虚线位置表示“当前decoding时刻”，beam width = 4。图中所有的block皆为逻辑块。

因为beam width = 4，这意味着每次decoding，都要找出top 4个最可能的next token，经过我们的计算，top 4 next token，有2个来自beam candidate 1，有2个来自beam candidate 2，因此我们在block6中分出block9和block10，用于装其中两个top 2 next token：对block7也是同理。

现在，block9/10/11/12中装的top 4 next token，就成为新的beam candidate。可以按照和上述步骤一样的方式继续做beam search算法，而对于block5和block6，它们已经在beam search的搜索算法中被淘汰了，后续生成的token也不再和它们产生关系，所以可以清除掉这两个逻辑块，并释放它们对应的物理块的内存空间。

CSDN @不秃不帅

对，我们继续往左边来看这幅图，block3引出block5/6/7，和block4引出block8/9，这意味着当前这4个top4token，是上一个timestep下candidate1和candidate3相关所对应的candidate1的2和2的block没有漏出，是因为它们所在的逻辑块beam search算法淘汰了，因此没有画出的必要），由于block3已经被淘汰，所以block4也相继被淘汰，并释放对应的物理内存空间。

由此往左一路推，直到block0为止（即block0代表prompt，因此就是beam search中所有的序列头象），这一路上，我们都根据最新时刻的beam search decoding结果，释放掉不再被需要的逻辑块和对应的物理内存空间，达到节省显存的目的。

五、调度和抢占

到目前为止，我们已经回答了“VLLM是如何优化KV cache显存分配”的问题，现在我们来回答另一个重要的问题：

- 当采用动态分配显存的办法时，虽然前面上同一时刻能处理更多的prompt了，但因为没有为每个prompt预留所足够的显存空间，如果在某一时刻整个显存被打满了，而此时所有的prompt都没做完推理，那该怎么办？

5.1 总原则

当有一堵请求来到VLLM服务器上时，VLLM需要一个调度原则来安排如何执行这些请求，这个调度原则概括如下：

- 先来的请求先被服务（First-Come-First-Serve，FCFS）
- 如有抢占的需要，后来的请求先被抢占（preemption）

（1）先来的请求先被服务

这个很好理解，当有一堵请求到达VLLM服务器时，VLLM肯定优先处理来得早的请求

（2）后来的请求先被抢占

想象一下，当一堵请求来到VLLM服务器的瞬间，导致gpu显存不足时，VLLM会怎么做呢？

最直接的办法，VLLM暂停这堵请求中最后到达的那批请求的推理，同时将它们相关的KV cache从gpu上释放掉，以便为更早到达的请求腾出足够的gpu显存，让它们完成推理任务，如果不这样做的话，各个请求间相互争夺gpu资源，最终将导致谁都没有任何一个请求能完成推理任务，于是乎，VLLM会暂停这批请求，以为gpu上有足够的空间了，就能恢复那些被中断的请求的执行了。

在资源不足的情况下，暂时中断一些任务的执行，这样的举动就被称为“抢占（preemption）”。

5.2 抢占和恢复被抢占的请求

对于这些因gpu资源不足而被抢占的任务，VLLM要完成两件事：

- 暂停它们的执行，同时将与之相关的KV cache从gpu上释放掉
- 等gpu资源充足时，重新恢复它们的执行

针对这两件事，VLLM分别设计了Swapping*（交换策略）和Recomputation*（重计算策略）来解决，我们来细看这两个策略：

（1）Swapping

对于被抢占的请求，VLLM要将其KV cache从gpu上释放掉，那么：

- 问题1：该释放哪些KV cache？
- 问题2：要把这些KV cache释放到哪里去？

先看问题1，由前文PagedAttention原理可知，一个请求可以拆成多个block，我们既可以选择释放掉部分block，也可以选择释放掉全部block，或者更科学些，我们可以预测一下哪些block被使用的频率最低，然后释放掉这些低频block（但这种方式实现起来难度较大，性价比不是很高），在VLLM中，采取的是all-or-nothing策略，即释放被抢占请求的所有block。

再来看问题2，对于这些被选中要释放的KV block，如果将它们直接丢弃，那未免过于浪费，VLLM采用的解决思路是将其从gpu上交换（Swap）到cpu上。这样等到gpu显存充份时，再把这些block从cpu上重载回来。

（2）Recomputation

知道了Swapping机制，重计算的过程也很好理解了：对于有抢任务（比如prefill sampling中并行采样数n=1的任务），当它们因为资源不足而被抢占时，可以不做swap，而直接释放它们的KV cache，把它们重新放入等待处理的队列中，等后续资源充足时再重新从prefill阶段开始做推理

对，到这里，我们总结一下VLLM对请求的调度处理流程：

- 当一堵请求来到VLLM服务器上时，按照First-Come-First-Serve（FCFS）原则，优先处理那些最早到来的请求。
- 当gpu资源不足时，为了保证这些请求能尽快做完推理，VLLM会对那些后到来的请求执行“抢占”，即暂时终止它们的执行。
- 一旦VLLM决定执行抢占操作，它会留存处理到到来的请求，在此期间，它会将其抢占的请求相关的KV block全部交换（swap）到cpu上，等交换完成后，VLLM才会继续处理到到来的请求。
- 部分情况下，对于一些请求seq，vllm会选择只把seq KV cache，将它重新放入等待队列中，后续重新做prefill

Figure 4. VLLM system overview

沙罗 字节跳动

在LLM推理实操中，某些场景下单卡是完成不了推理的，需要多卡，那么对于多gpu这种更普适性的情况，VLLM是怎么处理的呢？

上图显示了在分布式场景下，VLLM的整体运作流程：

- 首先，VLLM有一个中央调度器（Scheduler），它负责计算和管理每张卡上KV cache从逻辑块到物理块的映射表（block tables）
- 在做分布式计算时，Scheduler会将映射表广播到各张卡上，每张卡上的Cache engine接收到相关信息后，负责管理各卡上的KV block

上图中给出的例子，是用张量模型并行（megatron-lm）做分布式推理时的情况，对应图中每个worker上写的是model shard，在张量并行中，各卡上的输入数据相同，只是各个负责计算的model shard的KV cache，所以这种情况下，各卡上的逻辑块-物理块的映射关系其实是相同的（和的同一张block table），只是各卡上物理块中实际存储的数据不同而已。

编辑于 2025-08-06 17:56 · 北京

推荐阅读

图解大模型计算加速系列：VLLM源码解析3，块管理器BlockManag...
猛猛

手撕代码-高性能算子开发 Triton（Triton）
柚子君

面试必问的tcp还是一知半解？最浅显易懂的两并发架构TCP知识详解
linu... 发表于 linux服务器开发

0x06：SICP的魔法：并发、时间与流模拟
lfkdsk

加学 App内打开

觉得有帮助吗？ 0 0 0